

Proyecto de software pedagógico en Python para Bioestadística

Instrucciones, entregables y rúbrica de evaluación

🎯 Propósito

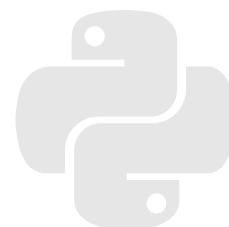
Diseñar, programar y defender un software en Python que explique de manera pedagógica un tema de Bioestadística, lo aplique correctamente a un ejercicio y permita evidenciar que el estudiante comprende tanto el código como el fundamento estadístico.

★ Nota máxima

La calificación máxima de esta actividad es **1.0**. Esta nota podrá aplicarse al parcial que el estudiante elija entre el **Parcial 1**, **Parcial 2**, **Parcial 3** o **Parcial 4**. Un trabajo perfecto obtiene **1.0**; a partir de allí se descuentan puntos según la rúbrica. La nota final se calcula como:

$$\text{Nota final} = \max\{0, 1.0 - \text{deducciones totales}\}.$$

Asignatura:	Bioestadística
Modalidad:	Trabajo 100% individual
Producto central:	Software funcional en Python con intención pedagógica
Evaluación:	Código, tema, ejercicio aplicado, defensa y aporte pedagógico
Aplicación de la nota:	La nota obtenida, hasta 1.0 , se aplicará al parcial elegido por el estudiante entre los parciales 1, 2, 3 o 4



1. Descripción general de la actividad

Cada estudiante debe desarrollar de manera **100% individual** un software en **Python** que enseñe, simule, calcule o ilustre uno de los temas vistos en clase. El software no debe limitarse a entregar una respuesta numérica: debe ayudar a comprender el concepto, guiar al usuario y mostrar con claridad cómo se llega al resultado.

Idea central

Un buen proyecto combina tres componentes: **rigor estadístico**, **código comprensible** y **valor pedagógico**. El estudiante debe poder explicar qué hace el programa, por qué lo hace y cómo se interpreta el resultado en contexto bioestadístico.

2. Temas permitidos

El proyecto debe centrarse en **uno** de los siguientes temas. Se permite integrar temas relacionados si esto mejora la explicación, pero debe existir un tema principal claramente identificado.

No.	Tema principal
1	Probabilidad conjunta, probabilidad condicionada y axiomas de probabilidad.
2	Teorema de Bayes y actualización de probabilidades con evidencia.
3	Distribuciones discretas: Bernoulli, binomial, Poisson, geométrica u otras vistas en clase.
4	Distribuciones continuas: normal, t de Student, chi-cuadrado, F u otras vistas en clase.
5	Pruebas de hipótesis: formulación de hipótesis, estadístico de prueba, valor p, región crítica e interpretación.
6	Razón de prevalencias (RP) con intervalos de confianza e interpretación epidemiológica.

3. Producto esperado

El software puede presentarse como una aplicación, cuaderno interactivo o programa ejecutable. Son opciones válidas, entre otras:

- un cuaderno de Jupyter o Google Colab bien organizado;
- una aplicación con Streamlit, Gradio, Dash u otra interfaz sencilla;
- un programa de consola con entradas, salidas y explicaciones claras;
- una simulación interactiva o visualización pedagógica en Python.

Requisitos mínimos del software

El software debe cumplir todos estos requisitos:

1. Identificar claramente el tema elegido y el objetivo de aprendizaje.
2. Recibir datos, parámetros o un ejercicio propuesto por el usuario.
3. Ejecutar correctamente los cálculos o simulaciones correspondientes.
4. Mostrar una explicación paso a paso del procedimiento estadístico.
5. Entregar una interpretación final en lenguaje comprensible para Bioestadística.

6. Incluir al menos una visualización, tabla, simulación o recurso que mejore el aprendizaje.
7. Estar documentado de forma que el estudiante pueda explicar su funcionamiento.

4. Entregables

El estudiante debe entregar una carpeta comprimida o repositorio con la siguiente estructura recomendada:

```
proyecto_bioestadistica/
|-- app.py # o notebook.ipynb
|-- README.md # explicacion general y modo de uso
|-- requirements.txt # librerias necesarias
|-- ejemplos/ # ejercicios o datos de prueba
|-- pruebas/ # verificaciones del resultado
`-- informe_breve.pdf # opcional, si el docente lo solicita
```

Contenido mínimo del README

El archivo README.md debe explicar: tema elegido, objetivo pedagógico, librerías usadas, instrucciones de instalación y ejecución, ejemplo de uso, interpretación esperada de los resultados y limitaciones del programa.

5. Defensa oral o sustentación

Durante la sustentación, el estudiante debe demostrar que entiende el trabajo. La defensa no consiste únicamente en mostrar que el programa funciona; debe evidenciar comprensión.

Aspecto	Lo que debe poder explicar el estudiante
Código	Estructura del programa, funciones principales, entradas, salidas, validaciones y librerías utilizadas.
Tema estadístico	Definiciones, supuestos, fórmulas, procedimiento y significado de los resultados.
Ejercicio aplicado	Datos usados, pasos del cálculo, resultado obtenido y conclusión bioestadística.
Aporte pedagógico	Cómo el software ayuda a aprender mejor que una solución puramente manual o una calculadora.
Limitaciones	Casos en los que el programa no aplica, supuestos no cumplidos o posibles mejoras.

6. Criterios de un trabajo perfecto

Un trabajo perfecto obtiene **1.0** si cumple simultáneamente lo siguiente:

1. El software ejecuta sin errores y resuelve correctamente el ejercicio propuesto.
2. El estudiante explica con claridad el código, las funciones y las decisiones de programación.
3. El estudiante domina el tema estadístico y lo comunica con lenguaje apropiado.
4. El programa tiene una intención pedagógica visible: explica, guía, compara, visualiza o retroalimenta.

5. Los resultados se interpretan correctamente en contexto bioestadístico.
6. La entrega es reproducible: cualquier evaluador puede instalar, ejecutar y verificar el programa.
7. La presentación es ordenada, ética y coherente con lo visto en clase.

7. Rúbrica de evaluación por descuentos

La nota parte de **1.0**. Se descuentan puntos por errores, omisiones o debilidades. En cada dimensión no se debe descontar más que el máximo indicado, salvo que aplique una condición de tope de nota de la sección 8.

Dimensión	Descuento máximo	Criterios de descuento
Comprensión conceptual del tema	-0.20	Errores conceptuales mayores: hasta -0.08 cada uno. Errores menores de definición o notación: hasta -0.03 cada uno. No explicar supuestos: hasta -0.05. Interpretar mal probabilidades, valores p, intervalos de confianza o RP: hasta -0.08. No conectar el resultado con Bioestadística: hasta -0.04.
Comprensión y explicación del código	-0.20	No poder explicar funciones, variables o flujo del programa: hasta -0.10. Código copiado o generado sin comprensión demostrada: hasta -0.15. Uso de librerías como “caja negra” sin explicar qué calculan: hasta -0.06. Código desordenado, sin nombres claros o sin comentarios mínimos: hasta -0.05.
Correctitud del software en el ejercicio	-0.20	Resultado incorrecto en el ejercicio evaluado: hasta -0.12. Fórmulas mal implementadas: hasta -0.10. No validar entradas o aceptar datos imposibles sin advertencia: hasta -0.04. No reproducir el resultado manual o teórico esperado: hasta -0.06. Fallos de ejecución durante la evaluación: hasta -0.10.
Aporte pedagógico	-0.15	El software solo calcula y no enseña: hasta -0.08. No incluye explicación paso a paso: hasta -0.05. No incluye ejemplos, visualizaciones, simulaciones, tablas comparativas o retroalimentación: hasta -0.05. Lenguaje confuso para el público objetivo: hasta -0.04.
Usabilidad e interfaz	-0.10	Instrucciones de uso incompletas: hasta -0.04. Interfaz confusa o difícil de usar: hasta -0.04. Salidas mal rotuladas o sin unidades/contexto: hasta -0.03. No informar errores al usuario: hasta -0.03.
Reproducibilidad y entrega	-0.10	No incluir <code>requirements.txt</code> o instrucciones de instalación: hasta -0.04. Archivos incompletos o desordenados: hasta -0.04. El evaluador no puede ejecutar el proyecto por falta de archivos: hasta -0.08. No incluir ejemplos o datos de prueba: hasta -0.03.
Comunicación, presentación y ética	-0.05	Presentación oral desorganizada: hasta -0.03. Informe o README con redacción deficiente que impide comprender el proyecto: hasta -0.03. No citar fuentes, datos o ayudas externas relevantes: hasta -0.04.
Total posible de descuentos	-1.00	La nota final no puede ser menor que 0.

8. Topes de nota por condiciones críticas

Algunas situaciones afectan la evaluación global porque impiden verificar el objetivo principal del trabajo.

⚠ Condiciones críticas

- Si no se entrega un software ejecutable, la nota máxima será **0.40**, aunque exista explicación teórica.
- Si el software ejecuta pero no corresponde a ningún tema permitido, la nota máxima será **0.50**.
- Si el estudiante no puede explicar el código durante la sustentación, la nota máxima será **0.60**.
- Si el estudiante no puede explicar el tema estadístico central, la nota máxima será **0.60**.
- Si el programa falla completamente durante la evaluación y no puede verificarse ningún resultado, la nota máxima será **0.50**.
- Si se evidencia plagio, suplantación o entrega de código sin autoría académica verificable, la calificación podrá ser **0.0**, según las normas institucionales.

9. Ejemplo de aplicación de la rúbrica

Suponga que un proyecto sobre teorema de Bayes funciona correctamente, pero el estudiante comete un error menor en la explicación conceptual, no incluye visualización y el README no explica cómo instalar dependencias. Un posible descuento sería:

$$\begin{aligned}\text{Nota final} &= 1.0 - 0.03 - 0.04 - 0.04 \\ &= 0.89.\end{aligned}$$

La asignación exacta del descuento depende de la gravedad, frecuencia y efecto del error sobre el aprendizaje y la validez estadística.

10. Lista de verificación para estudiantes

Antes de entregar, revise que pueda marcar todos los puntos:

☑ Checklist final

- Elegí un tema permitido y lo indiqué claramente.
- Mi software ejecuta sin errores desde cero.
- Incluí instrucciones de instalación y uso.
- Probé el programa con al menos un ejercicio y verifiqué el resultado.
- El programa explica los pasos, no solo entrega una respuesta final.
- Puedo explicar cada función importante del código.
- Puedo explicar el fundamento bioestadístico del procedimiento.
- Incluí interpretación de resultados en contexto.
- Incluí visualización, simulación, tabla o recurso pedagógico útil.
- Reconocí las fuentes y ayudas utilizadas.

11. Recomendaciones por tema

Tema	Ideas de software pedagógico
Probabilidad conjunta y condicionada	Calculadora con tablas de contingencia, diagramas de Venn, simulación de eventos y verificación de axiomas.
Teorema de Bayes	Herramienta para actualizar probabilidades ante una prueba diagnóstica, con sensibilidad, especificidad, prevalencia y valores predictivos.
Distribuciones discretas	Simulador de ensayos binomiales o eventos Poisson, comparando probabilidad teórica y frecuencia simulada.
Distribuciones continuas	Visualizador de curvas, áreas bajo la curva, puntajes z, percentiles e interpretación de probabilidades.
Pruebas de hipótesis	Asistente que formule hipótesis, calcule estadístico, valor p y ayude a decidir con una interpretación no mecánica.
RP con intervalos de confianza	Calculadora para estudios transversales, tabla 2x2, RP, intervalo de confianza e interpretación epidemiológica.

El objetivo no es solamente programar: es usar Python para aprender, explicar y aplicar Bioestadística con rigor.